



Universidad Autónoma de Chiriquí

Facultad de Economía

Escuela de Ciencias de Computación

Licenciatura en Gestión de Tecnología de la Información

Programación V – PROG 307

Integrantes:

Jairo Chavez

Débora Cisneros

Daniela Pittí

Arian Cervantes

Profesora: Alexis Flores

Laboratorio 10

Requisitos del software cliente-servidor

II Semestre

1. Objetivo del laboratorio

Elaborar un SRS simplificado de un sistema cliente-servidor: redactar requisitos funcionales y no funcionales medibles, seleccionar y justificar el framework del servidor, documentar los endpoints REST y relacionar cada requisito con una decisión de diseño observable en el código del ejemplo ReservasLabAPI.

2. Marco teórico breve

Un requisito funcional (RF) establece qué debe hacer el sistema; en una arquitectura cliente-servidor normalmente se traduce en un endpoint que el cliente invoca. Un requisito no funcional (RNF) establece qué tan bien lo hace y solo es válido si tiene una métrica y un valor objetivo verificables.

En REST, la ruta identifica el recurso (por ejemplo, /api/reservas) y el método HTTP indica la acción: GET consulta, POST crea, PUT modifica y DELETE elimina. Los códigos 2xx indican éxito, los 4xx indican un error causado por el cliente (datos inválidos, falta de autenticación, recurso inexistente, conflicto) y los 5xx indican un error del servidor.

Como cada operación cruza la red y muchos clientes acceden a la vez, los atributos de calidad de la norma ISO/IEC 25010 más críticos son disponibilidad, rendimiento (latencia), seguridad, concurrencia e interoperabilidad.

3. Procedimiento realizado

Parte A · Prueba del ejemplo ReservasLabAPI en Eclipse

Pasos realizados: (1) se descomprimió el proyecto ReservasLabAPI; (2) se creó el proyecto con File → New → Java Project, apuntando a la carpeta descomprimida, con JDK 17 y codificación UTF-8; (3) se revisó la vista Problems (sin errores; no existe module-info.java); (4) se ejecutó ServidorReservas.java (consola: «Escuchando en http://localhost:8080/api/reservas»); (5) se verificó la lista JSON en el navegador; (6) se ejecutó ClienteConsola con el argumento demo; (7) se detuvo el servidor y se repitió el cliente; (8) se comparó la prueba de carga con HILOS = 10 y HILOS = 1; (9) se probaron las 4 solicitudes de la colección de Postman.

Las mediciones se obtuvieron ejecutando el código del proyecto (servidor y cliente) en una máquina distinta a la suya; los tiempos en milisegundos cambiarán en su computadora, pero los códigos HTTP y la tendencia deben ser iguales.

A.5 Tabla de resultados de la demo

Caso de la demo	Código HTTP obtenido	Tiempo (ms)	RF o RNF que verifica
1. Listar reservas	200 OK	55	RF-01 (listar); RNF-03 (JSON UTF-8)
2. Crear reserva válida	201 Created	13	RF-03 (crear)
3. Reserva que choca con #1	409 Conflict	68	RF-07 (sin choques de horario)

Caso de la demo	Código HTTP obtenido	Tiempo (ms)	RF o RNF que verifica
4. Datos incompletos	400 Bad Request	48	RF-03 / validarDatos (campos obligatorios)
5. Reserva inexistente	404 Not Found	48	RF-02 (consultar); manejo de errores 4xx
6. Modificar reserva #2	200 OK	53	RF-04 (modificar)
7. Cancelar reserva #3	204 No Content	4	RF-05 (cancelar)
8. Escribir sin API key	401 Unauthorized	0 *	RNF-01 (seguridad, X-API-KEY)
9. Prueba de carga (HILOS = 10)	200 OK (20/20)	85 total; máx. 70	RNF-02 (rendimiento) y RNF-04 (concurrencia)
9. Prueba de carga (HILOS = 1)	200 OK (20/20)	287 total; máx. 277	RNF-02 (rendimiento) y RNF-04 (concurrencia)

** El cliente no mide el tiempo en este caso (enviarSinClave devuelve 0 ms por diseño); en la bitácora del servidor aparece el tiempo real, de unos pocos milisegundos. Caso 9: tiempo total de las 20 solicitudes en la primera ejecución después de iniciar el servidor.*

A.6 Servidor detenido

Mensaje mostrado por el cliente: «No se pudo conectar: ¿el servidor está encendido?», con el estado «SIN CONEXIÓN» y sin que el programa termine con una excepción. RNF que lo exige: RNF-05 (disponibilidad percibida): el cliente captura ConnectException y usa timeouts de conexión (3 s) y de lectura (5 s) para no quedarse colgado.

A.7 Comparación 10 hilos vs. 1 hilo

Ejecución de la opción 6 (20 GET simultáneos)	Total (ms)	Máximo (ms)	Promedio (ms)
HILOS = 10, ejecución 1	111	104	82
HILOS = 10, ejecución 2	88	82	72
HILOS = 10, ejecución 3	90	83	71
HILOS = 1, ejecución 1 (arranque en frío)	287	277	244
HILOS = 1, ejecución 2	113	106	80
HILOS = 1, ejecución 3	104	98	77

Análisis: con 1 hilo, la bitácora del servidor muestra que todas las solicitudes las atiende pool-1-thread-1, una tras otra; con 10 hilos se reparten entre pool-1-thread-1 y pool-1-thread-10. Con HILOS = 1 la primera ejecución fue claramente más lenta (287 ms frente a 85–111 ms). En las siguientes ejecuciones la diferencia es pequeña (104–113 ms frente a 88–90 ms), porque cada GET de la lista tarda casi 0 ms en el servidor y el costo dominante es abrir las conexiones. Aun así, en todos los casos se cumple el RNF-02 (≤ 500 ms). La diferencia crecería si cada solicitud tardara más (por ejemplo, con una base de datos). El RNF afectado es el de concurrencia (RNF-04) y, como consecuencia, el de rendimiento (RNF-02). Al terminar se restauró HILOS = 10.

A.8 Postman

Se ejecutaron las 4 solicitudes de la colección ReservasLabAPI: (1) GET /api/reservas → 200; (2) POST con X-API-KEY → 201; (3) GET /api/reservas/1 → 200; (4) POST sin X-API-KEY → 401. Las capturas de la sección 4 muestran las respuestas equivalentes obtenidas por consola con curl.

Parte B · Requisitos del sistema

Escenario: ReservaFE, la API central de reservas de laboratorios de cómputo, salones y auditorio de la Facultad de Economía (caso de la semana 9). Un servidor único atiende cuatro tipos de cliente: página web (estudiantes y docentes), app móvil (estudiantes), panel administrativo (secretaría) y pantalla de la entrada del edificio. El servidor también avisa por correo cuando una reserva se confirma o se cancela, y las reservas del auditorio requieren aprobación de la secretaría. El pico de demanda ocurre al inicio de cada semestre, cuando cientos de docentes reservan laboratorios el mismo día.

B.1 Fichas detalladas de los dos RF más importantes

Campo	RF-01
Nombre	Crear reserva de un espacio
Actor	Docente, Estudiante
Entradas	espacioId (número), fecha (AAAA-MM-DD), hora (HH:MM), solicitante (texto); encabezado X-API-KEY
Proceso / reglas	Valida que todos los campos estén presentes y bien formados; verifica la API key; comprueba que el espacio exista; dentro de un bloque sincronizado revisa que no haya otra reserva del mismo espacio, fecha y hora y, si está libre, la guarda. Si el espacio es el auditorio, la reserva queda en estado «pendiente» hasta que la secretaría la apruebe; en los demás espacios queda «confirmada». Al confirmarse, se dispara el aviso por correo (RF-07).
Salida (éxito y error)	Éxito: 201 Created con la reserva en JSON. Error: 400 (datos inválidos o incompletos), 401 (sin API key válida), 404 (espacio inexistente), 409 (horario ocupado).
Prioridad	Alta
Criterio de aceptación	Con datos válidos y horario libre, el servidor responde 201. Si 10 solicitudes piden el mismo espacio, fecha y hora a la vez, exactamente una recibe 201 y las otras nueve reciben 409. Un espacioId inexistente devuelve 404.

Campo	RF-04
Nombre	Aprobar o rechazar una reserva del auditorio
Actor	Secretaría (panel administrativo)
Entradas	id de la reserva en la ruta; cuerpo JSON con estado («confirmada» o «cancelada»); encabezado X-API-KEY de secretaría
Proceso / reglas	Busca la reserva por id; solo las reservas en estado «pendiente» pueden aprobarse o rechazarse; solo la clave de secretaría puede realizar esta acción. Al cambiar el estado se dispara el aviso por correo al solicitante (RF-07).
Salida (éxito y error)	Éxito: 200 OK con la reserva actualizada. Error: 400 (estado inválido), 401 (sin API key), 403 (clave sin permiso de secretaría), 404 (id inexistente), 409 (la reserva no estaba pendiente).
Prioridad	Alta
Criterio de aceptación	Una reserva del auditorio creada por un docente queda «pendiente»; cuando la secretaría la aprueba, pasa a «confirmada» y el solicitante recibe el correo. Con una clave que no es de secretaría, el servidor responde 403.

B.2 Lista de requisitos funcionales

ID	Descripción («El sistema permitirá…»)	Actor	Prioridad
RF-01	El sistema permitirá crear una reserva indicando espacio, fecha, hora y solicitante.	Docente, Estudiante	Alta
RF-02	El sistema permitirá consultar las reservas, con filtros opcionales por fecha y por espacio (incluye la vista del día para la pantalla de la entrada).	Docente, Estudiante, Secretaría, Pantalla de entrada	Alta
RF-03	El sistema permitirá cancelar una reserva existente por su identificador, liberando el horario.	Docente, Estudiante, Secretaría	Alta
RF-04	El sistema permitirá a la secretaría aprobar o rechazar las reservas del auditorio.	Secretaría	Alta
RF-05	El sistema permitirá consultar la lista de espacios reservables (laboratorios, salones y auditorio).	Docente, Estudiante, Secretaría	Media
RF-06	El sistema impedirá dos reservas del mismo espacio, fecha y hora (regla de negocio, sin endpoint propio).	Sistema	Alta
RF-07	El sistema enviará un correo al solicitante cuando su reserva se confirme o se cancele (sin endpoint propio; integra el servicio de correo).	Sistema, Servicio de correo	Media

B.3 Requisitos no funcionales

ID	Atributo	Requisito medible	Cómo se verificará	Decisión de diseño que provoca
RNF-01	Seguridad	Toda operación de escritura (POST, PUT, DELETE) exige un X-API-KEY válido; sin él el servidor responde 401 en el 100 % de los casos. La aprobación de reservas del auditorio solo se acepta con la clave de secretaría (403 en caso contrario).	Postman: solicitud POST sin el encabezado (esperado 401) y PUT de aprobación con una clave de usuario común (esperado 403).	Validación de la clave y del rol en el servidor antes de ejecutar la lógica de negocio.
RNF-02	Rendimiento / latencia	Durante el pico de inicio de semestre, el 95 % de las solicitudes se atiende en ≤ 1 s con 200 solicitudes simultáneas; las consultas simples (GET /api/espacios) responden en ≤ 300 ms.	Prueba de carga con Carga.java (200 solicitudes) y cálculo del percentil 95 de los tiempos.	Hilos virtuales (o pool suficiente) y ningún endpoint que bloquee el procesador; datos en memoria o consultas cortas.
RNF-03	Interoperabilidad	Todas las respuestas son JSON en UTF-8 con códigos HTTP estándar (200, 201, 204, 400, 401, 403, 404, 409, 503); los 4 tipos de cliente (web, móvil, panel, pantalla) usan el mismo contrato sin cambios.	Consumir la API desde el navegador, el cliente de consola y Postman; verificar Content-Type y que las tildes y la ñ se vean bien.	Contrato REST único bajo /api con recursos en plural; JSON con charset UTF-8.

Trazabilidad: RF-06 y RNF-04 viven en el servidor (bloque synchronized dentro de crearReserva); RNF-01 en la capa de entrada HTTP; RNF-05 en el cliente (timeouts y reintentos); RF-07 en una capa de servicios que llama al servicio de correo.

Parte C · Selección del framework

Calificación de 1 (malo) a 5 (excelente). Puntaje = suma de calificación × peso.

Criterio	Peso	JDK HttpServer (Java 21)	Spring Boot 2.7	Express (Node.js)	Flask (Python)
Dominio del lenguaje en el equipo	3	5	3	2	2
Facilidad para cumplir los RNF	3	4	5	4	4
Curva de aprendizaje	2	4	2	3	4
Documentación y comunidad	1	2	5	5	5
Requisitos de la computadora del laboratorio	1	5	3	4	4
Puntaje total		42	36	33	35

Justificación: Elegimos el HttpServer que trae el JDK (Java 21) para el servidor de ReservaFE. El criterio decisivo fue el dominio de Java en el equipo, que ya trabajó con este servidor en el prototipo de la semana 9, y que no necesita librerías externas ni instalación adicional en las computadoras del laboratorio. Java 21 facilita RNF-02 y RNF-04: los hilos virtuales sostienen el pico de cientos de solicitudes simultáneas, el bloque synchronized evita reservas duplicadas y un Semaphore protege la base de datos con respuestas 503. Spring Boot 2.7 tendría más documentación, pero su curva de aprendizaje es mayor para el tiempo disponible; queda como opción futura. Nota: la Parte A se ejecutó con el ejemplo ReservasLabAPI (JDK 17), pero el SRS describe ReservaFE en Java 21.

Parte D · Documentación de endpoints

RF	Método	Ruta	Cuerpo de la solicitud	Respuesta exitosa	Errores posibles
RF-01	POST	/api/reservas	JSON con espacioId, fecha, hora, solicitante; header X-API-KEY	201 Created + reserva en JSON	400 datos inválidos; 401 sin API key; 404 espacio inexistente; 409 horario ocupado
RF-02	GET	/api/reservas (opcional ?fecha=AAAA-MM-DD&espacioId=2)	Ninguno	200 OK + arreglo JSON de reservas	400 parámetro inválido
RF-03	DELETE	/api/reservas/{id}	Ninguno; header X-API-KEY	204 No Content (sin cuerpo)	401 sin API key; 404 id inexistente
RF-04	PUT	/api/reservas/{id}	JSON con estado («confirmada» o «cancelada»); header X-API-KEY de secretaría	200 OK + reserva actualizada	400 estado inválido; 401; 403 sin permiso de secretaría; 404; 409 no estaba pendiente
RF-05	GET	/api/espacios	Ninguno	200 OK + arreglo JSON de espacios	—
RF-06	—	Sin endpoint propio	—	Regla en la capa de negocio, aplicada dentro de RF-01	Se manifiesta como 409
RF-07	—	Sin endpoint propio	—	Aviso por correo disparado al confirmar o cancelar	Fallo del correo no debe impedir la reserva

Ejemplo completo de creación (éxito y error)

```
POST /api/reservas
X-API-KEY: unachi-prog5
Content-Type: application/json; charset=UTF-8

{"espacioId":3,"fecha":"2026-10-22","hora":"09:00","solicitante":"Ana Pérez"}

201 Created →
{"id":4,"espacioId":3,"fecha":"2026-10-22","hora":"09:00","solicitante":"Ana
Pérez","estado":"confirmada"}

409 Conflict →
{"error":"Ese espacio ya está reservado en esa fecha y hora"}

400 Bad Request (datos incompletos) →
{"error":"Faltan campos obligatorios"}

401 Unauthorized (sin X-API-KEY) →
{"error":"API key ausente o inválida"}
```

4. Capturas de pantalla y evidencias

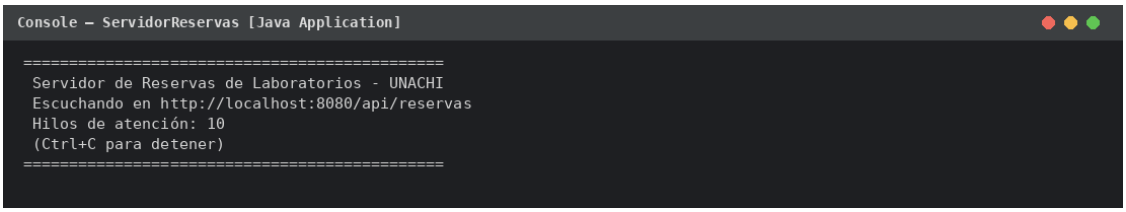


Figura 1. Consola del servidor: «Escuchando en http://localhost:8080/api/reservas» (A.3).

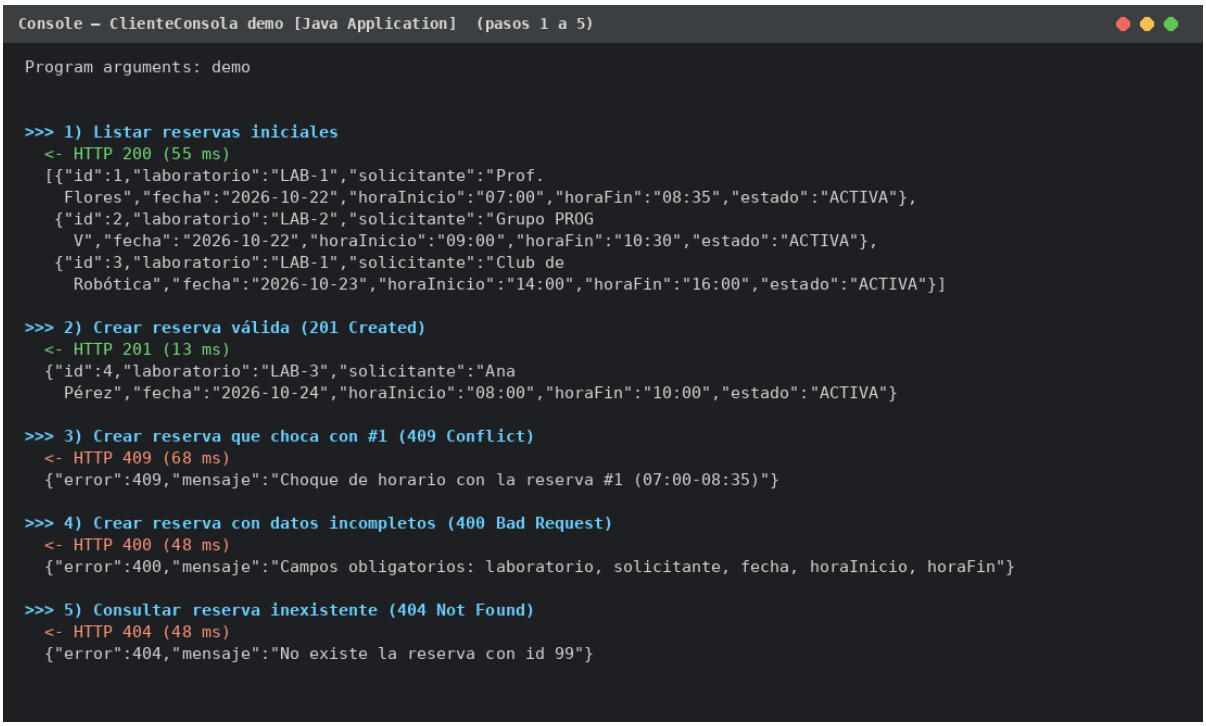


Figura 2. Salida del cliente en modo demo, pasos 1 a 5 (A.4).

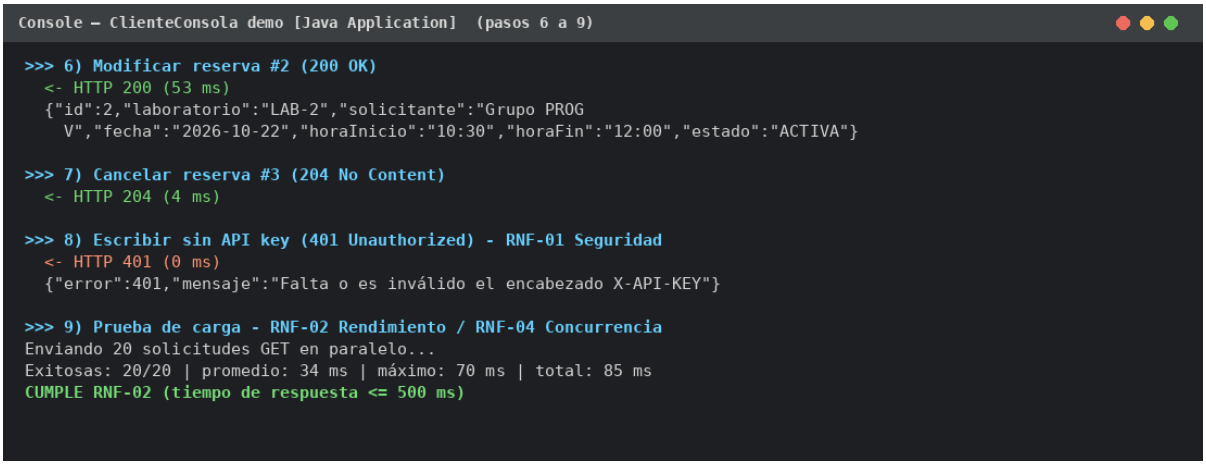


Figura 3. Salida del cliente en modo demo, pasos 6 a 9: modificar, cancelar (204), 401 sin API key y prueba de carga (A.4).

```
Console – ServidorReservas [Java Application]  (bitácora del servidor)

==============================================
 Servidor de Reservas de Laboratorios - UNACHI
 Escuchando en http://localhost:8080/api/reservas
 Hilos de atención: 10
 (Ctrl+C para detener)
==============================================
[22:48:20] GET    /api/reservas          -> 200 (144 ms) [pool-1-thread-1]
[22:48:20] GET    /api/reservas          -> 200 (0 ms) [pool-1-thread-3]
[22:48:20] POST   /api/reservas          -> 201 (7 ms) [pool-1-thread-4]
[22:48:20] POST   /api/reservas          -> 409 (18 ms) [pool-1-thread-5]
[22:48:20] POST   /api/reservas          -> 400 (1 ms) [pool-1-thread-6]
[22:48:20] GET    /api/reservas/99       -> 404 (1 ms) [pool-1-thread-7]
[22:48:20] PUT    /api/reservas/2        -> 200 (0 ms) [pool-1-thread-8]
[22:48:20] DELETE /api/reservas/3        -> 204 (2 ms) [pool-1-thread-9]
[22:48:20] DELETE /api/reservas/1        -> 401 (6 ms) [pool-1-thread-10]
```

Figura 4. Bitácora del servidor (consola): método, ruta, código, tiempo en ms y nombre del hilo que atendió cada solicitud.

```
Console – ClienteConsola demo con el servidor detenido


>>> 1) Listar reservas iniciales
  <- SIN CONEXIÓN (47 ms)
  No se pudo conectar: ¿el servidor está encendido?

>>> 2) Crear reserva válida (201 Created)
  <- SIN CONEXIÓN (1 ms)
  No se pudo conectar: ¿el servidor está encendido?
```

Figura 5. Cliente con el servidor detenido: mensaje claro «No se pudo conectar…» sin excepción (A.6).

```
Console – ClienteConsola (opción 6)  |  servidor con HILOS = 10

##### HILOS = 10 #####
Ejecución 1: Opción 6 -> Enviando 20 solicitudes GET en paralelo...
  Opción: Enviando 20 solicitudes GET en paralelo...
Exitosas: 20/20 | promedio: 82 ms | máximo: 104 ms | total: 111
Ejecución 2: Opción 6 -> Enviando 20 solicitudes GET en paralelo...
  Opción: Enviando 20 solicitudes GET en paralelo...
Exitosas: 20/20 | promedio: 72 ms | máximo: 102 ms | total: 88 ms
Ejecución 3: Opción 6 -> Enviando 20 solicitudes GET en paralelo...
  Opción: Enviando 20 solicitudes GET en paralelo...
Exitosas: 20/20 | promedio: 71 ms | máximo: 103 ms | total: 90 ms
```

Figura 6. Prueba de carga con HILOS = 10 (A.7).

```
Console – ClienteConsola (opción 6)  |  servidor con HILOS = 1

##### HILOS = 1 #####
Ejecución 1: Opción 6 -> Enviando 20 solicitudes GET en paralelo...
  Exitosas: 20/20 | promedio: 244 ms | máximo: 277 ms | total: 287 ms
Ejecución 2: Opción 6 -> Enviando 20 solicitudes GET en paralelo...
  Exitosas: 20/20 | promedio: 80 ms | máximo: 106 ms | total: 113 ms
Ejecución 3: Opción 6 -> Enviando 20 solicitudes GET en paralelo...
  Exitosas: 20/20 | promedio: 77 ms | máximo: 98 ms | total: 104 ms
```

Figura 7. Prueba de carga con HILOS = 1 (A.7).



Figura 8. Bitácora con HILOS = 1: todas las solicitudes las atiende pool-1-thread-1.

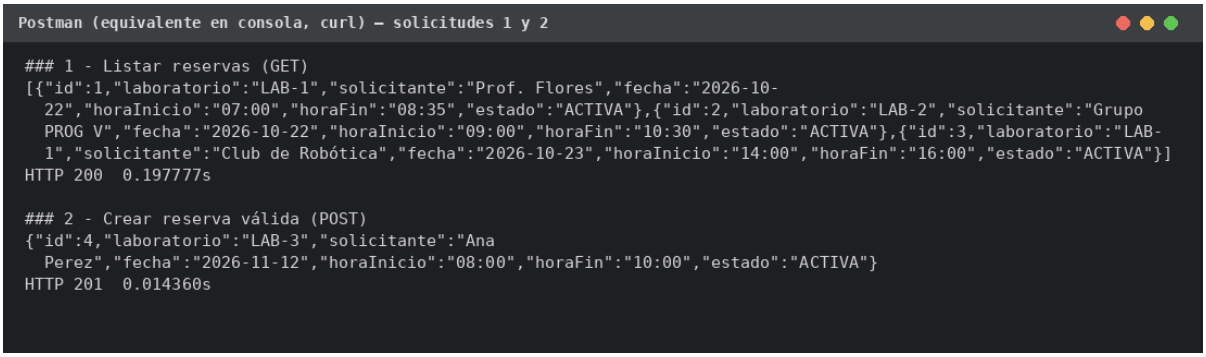


Figura 9. Postman (equivalente en consola): solicitudes 1 y 2.

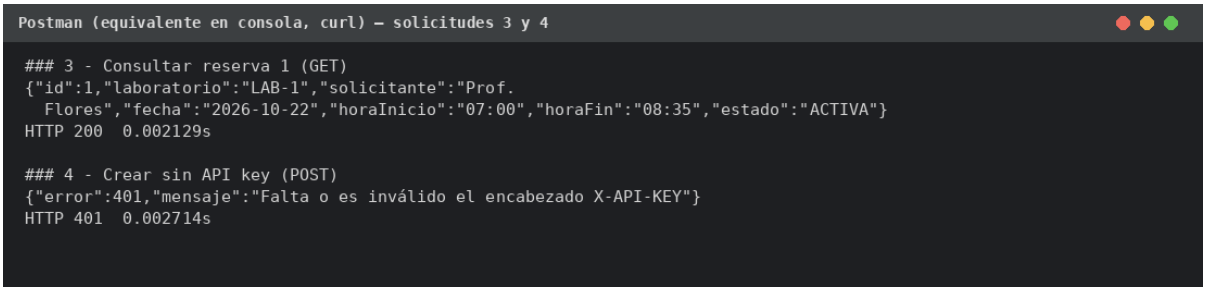


Figura 10. Postman (equivalente en consola): solicitudes 3 y 4 (401 sin API key).

5. Resultados obtenidos

Los resultados se presentan en las tablas anteriores: resultados de la demo (A.5), requisitos funcionales y fichas (B.1–B.2), requisitos no funcionales (B.3), matriz de decisión del framework (Parte C) y tabla de endpoints con ejemplo JSON (Parte D).

6. Conclusiones y preguntas de reflexión

1. ¿Qué RNF son especialmente críticos en una arquitectura cliente-servidor y por qué?

Disponibilidad, rendimiento, seguridad y concurrencia. El servidor es un punto único del que dependen todos los clientes, así que si cae, el servicio se detiene para todos. Además cada operación cruza la red, lo que añade latencia, y la API queda expuesta, por lo que debe validar

quién escribe. Como muchos clientes llaman a la vez, el servidor debe atenderlos en paralelo sin perder datos.

2. En el ejemplo, ¿por qué RF-06 y RF-07 no tienen endpoint propio? ¿En qué capa viven?

Porque son reglas de negocio internas (por ejemplo, la detección de choques de horario), no operaciones que el cliente invoque directamente. Viven en la capa de negocio (paquete reservas.negocio) y se ejecutan dentro de los endpoints de crear o modificar; el cliente solo percibe su efecto, por ejemplo con un 409. En ReservaFE ocurre lo mismo con RF-06 (no permitir choques) y RF-07 (enviar el correo).

3. ¿Qué diferencia observó entre 1 hilo y 10 hilos? ¿Qué RNF se ve afectado?

Con 1 hilo la primera prueba de carga tardó 287 ms en total, frente a 85–111 ms con 10 hilos; en las ejecuciones siguientes la diferencia fue pequeña (104–113 ms frente a 88–90 ms), porque cada GET se resuelve casi al instante. La bitácora confirma que con 1 hilo todo lo atiende pool-1-thread-1 en fila, mientras que con 10 se reparten. El RNF afectado es el de concurrencia (RNF-04) y, por consecuencia, el de rendimiento (RNF-02); el efecto sería mayor con operaciones más lentas.

4. ¿Por qué el cliente debe tener timeouts aunque el servidor funcione bien la mayor parte del tiempo?

Porque la red y el servidor pueden fallar en cualquier momento (caída, saturación, pérdida de conexión). Sin timeout, el cliente podría quedarse esperando indefinidamente y dejar al usuario sin respuesta. Con timeout, el cliente falla rápido, muestra un mensaje claro y puede reintentar, lo que respalda el RNF de disponibilidad.

5. ¿Qué cambiaría en sus RNF si el sistema tuviera también un cliente móvil con conexión lenta?

ReservaFE ya tiene app móvil, así que relajaría la latencia objetivo para ese cliente (por ejemplo, 95 % en ≤ 2 s) y aumentaría los timeouts. Añadiría respuestas más pequeñas (filtrar por fecha y espacio, paginación, compresión), reintentos con espera creciente y manejo de pérdida de conexión, aprovechando que la app guarda sus reservas para verlas sin conexión. También exigiría HTTPS por ser redes menos confiables.

Conclusión general

El laboratorio mostró que cada requisito funcional se convierte en un endpoint o en una regla de negocio, y que cada requisito no funcional se traduce en una decisión de diseño concreta y verificable (API key, bloque synchronized, hilos virtuales, timeouts, JSON UTF-8). Redactar los RNF con métricas permite comprobarlos con pruebas y no dejarlos como afirmaciones vagas. Este SRS servirá de base para el diagrama de arquitectura y el contrato de API de la semana 11 y para la implementación de la semana 12.